

Summing Numbers in an Array

Our first example is simple: given a list (in Python) or an array (in JavaScript) of integers, return the total sum of all the integers. For example, a call such as `sum([5, 2, 4, 8])` should return 19.

This is easy to solve with a loop, but solving it with recursion requires more thought. After reading Chapter 2, you might also notice that this algorithm doesn't map well enough to recursion's capabilities to justify recursion's added complexity. Still, summing numbers in an array (or some other calculation based on processing data in a linear data structure) is a common enough recursion problem in coding interviews that it deserves our attention.

To solve this problem, let's examine the *head-tail technique* for implementing recursive functions. This technique splits the recursive function's array argument into two parts: the *head* (the first element of the array) and the *tail* (a new array including everything after the first element). We define the recursive `sum()` function to find the sum of the array argument's integers by adding the head to the sum of the tail array. To find out the sum of the tail array, we recursively pass it as the array argument to `sum()`.

Because the tail array is one element smaller than the original array argument, we'll eventually end up calling the recursive function and passing it an empty array. An empty array argument is trivial to sum and doesn't require more recursive calls; it is merely 0. From these facts, our answers to the three questions are as follows:

What is the base case? An empty array, which has the sum of 0.

What argument is passed to the recursive function call? The tail of the original number array, which has one less number than the original array argument.

How does this argument become closer to the base case? The array argument shrinks by one element for each recursive call until it becomes a zero-length, or empty, array.

Here is *sumHeadTail.py*, a Python program to sum a list of numbers: